FusePlan – Presentation script

Separating Complementarity from Ensemble Effects in Partial Floor Plan Retrieval

How to use it. One section per slide, in the order of the deck. Each section has: what is on the slide; *what to say*, written in full so you understand what we did, where and why; and the questions that slide can raise. The only formulas are the paper's own (fusion, gain and CG, AUC, plus the pooling and whitening of the vision branch). Numbers follow the slides (AUC ×100, "points").

The thread, in the order of the deck. Partial-plan retrieval is a new task (slide 2) → we solve it with an image branch, a graph branch and a late fusion, plus an ensemble control (3) → the graph is strong and every fusion improves on it (4) → how much fusion helps depends on the relative strength of the branches (5) → the control shows what the image really adds (6) → what we conclude (7).

Suggested split (two slides each, the first speaker also opens with the title): slides 2–3, 4–5, 6–7. Slide 3 is the densest.

Slide 1 – Title

On the slide. FusePlan; the subtitle; the three authors, University of Modena and Reggio Emilia.

What to say. FusePlan retrieves floor plans from *incomplete* queries by fusing two views of a plan, its image and its graph of rooms. The subtitle is our question: when the fusion improves the result, how much of the gain is *complementarity* (the image brings information the graph lacks) and how much is an *ensemble effect* (any two models combined do better than one)?

Slide 2 – Problem and related work

On the slide. Left: problem, goal, question. Right: a table of related work, one line each, and the gap.

What to say

Problem. An architect rarely starts from a complete plan: they sketch a few rooms or reuse a fragment, then look for complete plans that contain it. Existing floor-plan retrieval methods train and evaluate on complete plans; we study retrieval from incomplete ones.

Goal. From a partial plan, find its source among the 67,405 plans of RPLAN. Concretely: we take a plan, remove part of its rooms, and the system must rank the whole collection so that the intact original comes as high as possible. We call this *self-recovery*: the right answer is known by construction, so no human relevance labels are needed.

Why two views, and why it needs care. A plan exists as an image and as a graph of rooms, and they fail differently when rooms are missing. In the image, a removed room leaves a hole inside the outline of the plan: the image still says "something of this shape was here". In the graph, the removed rooms simply disappear, and what is left looks like the complete graph of some smaller plan. So the image *might* add information. But combining any two models whose errors are partly independent improves the result, even if the second one knows nothing new: two trainings of the same network already make different mistakes.

Question. Does the image add information to the graph, or does it only act as a second model?

Related work (one sentence each, as on the slide):

- *LayoutGMN*: graph matching between two complete plans. *LayoutGKN*: graph kernel on complete plans; it is the closest graph method, and we retrain it on our task as baseline. *SSIG*, *Unit Region Encoding*: similarity of complete plans. All evaluate on complete plans.
- *Graph2Plan*, *PlanCraft*: partial plans appear, but for *generating* plans, not retrieving them.
- *On-the-fly sketch retrieval*: retrieves a photo while the sketch is being drawn, so the query is

incomplete; our AUC metric follows theirs.
- *CrossOver*, *MMFE*: multimodal representations learned end to end; they report only the fused accuracy, so they cannot tell new information from an ensemble effect.

Gap: no retrieval method takes partial plans as queries (in our literature search, not exhaustive).

If asked

Q: Why self-recovery and not "find similar plans"?

Similarity needs relevance labels that RPLAN does not have, and any score derived from the metadata is circular for our branches (slide 4). With self-recovery the correct answer is known, so the metric is objective.

Slide 3 – Method

On the slide. Pipeline: plan image → frozen PE-Spatial with GeM → PCA whitening → 768-d vector; room graph → GATv2 trained on damaged–complete plans → 128-d vector; late fusion with $\alpha = 0.4$, 896-d, one exact FAISS search over 67,405 plans. Below: graph branch, vision branch, complementarity gain.

What to say

Overall. Two independent branches turn every plan into a unit-length vector. Both index the same gallery of complete plans and receive the *same* removed rooms, so every system can be compared query by query. Similarity is the cosine; for unit vectors it equals the inner product, so we search with an exact inner-product FAISS index, which compares the query with every gallery plan. With 67k plans this is fast, and exact search leaves no approximation error in the comparisons.

Vision branch (top row). The plan's colour rendering is resized (longer side 224 pixels, aspect ratio kept) and padded to a square with the white background. It goes through **PE-Spatial**, the spatially aligned variant of Meta's Perception Encoder (a ViT-B/16), used *frozen*: no training. The encoder gives one vector per image patch; we pool them into one vector with **GeM**, a generalised mean computed per channel, $\left(\frac{1}{N}\sum_i \max(x_i, \epsilon)^p\right)^{1/p}$ with $p = 3$, which gives more weight to the strongest activations than a plain average. One detail: the $\max(\cdot, \epsilon)$ comes from GeM's original use on CNN features, which are never negative; ViT features are signed, so the clamp zeroes the negative values and the descriptor pools only the positive part of each channel. We kept it because it is the configuration that won on validation.

Then **PCA whitening**: $x \mapsto \Lambda^{-1/2}U^\top(x - \mu)$, with mean and covariance estimated on training plans only, keeping all 768 components, then L2 normalisation. In words: centre, rotate onto the principal axes, give every axis unit variance. It matters because every rendering has the same white background, black walls and few colours: without whitening all plans look alike to the encoder and a damaged query matches almost anything. Centring removes what all plans share; rescaling gives the small directions, where plans differ, the same weight as the large common ones. The result is a 768-d vector (768 is the width of a ViT-B).

Optional damage-trained heads. To test whether an image branch that *learns the damage* carries more information, we trained two small heads on the frozen vectors. They are *alternatives*: each replaces the frozen vector in its own fusion; they are never stacked, even if the slide labels read "+ PE-Spatial + SDH".

- **Single-damage head (SDH):** an MLP 768 → 1024 → 256, applied to queries and gallery, trained to map a plan with rooms removed onto its complete version; its output is 256-d, re-whitened.
- **Multi-damage head (MDH):** acts on the query only, adding a small learned correction to the frozen vector; the gallery stays frozen. Its last layer starts at zero, so at the start it *is* the frozen branch and moves away only where training helps. It trains on three damages (room removal, crop, patch).

Both are trained contrastively (pull a damaged view and its complete plan together, push the other plans of the batch apart), without relevance labels.

Why PE-Spatial. We compared 8 frozen encoders, 52 configurations (slide 5), chosen on validation. PE-Spatial with GeM and whitening was the most robust. *(Interpretation, not measured: in a flat synthetic drawing what matters is* where *things are, which an encoder trained for spatial features captures better.)*

Graph branch (bottom row). The graph is built from the RPLAN *record*, not from the image. Each room is a node, described by its type (13 types) and the geometry of its box (centre, width, height, area, aspect ratio). Each adjacency between rooms is an edge, with its type (10 types, e.g. door or wall). The encoder has three **GATv2** layers: at each layer every room collects information from its neighbours, weighting each neighbour with a learned attention score that depends on both rooms and on the type of their adjacency. Then we *sum* the room vectors into one plan vector, add the plain count of room types (a skip connection), project to 128 dimensions and normalise.

Why. sum, not mean: the sum keeps the number of rooms, a strong clue. The skip connection hands the room counts directly to the output; removing it hurt every encoder. GATv2 over GCN and GraphSAGE: same recipe for the three, GATv2 best, GraphSAGE close, GCN far behind (slide 5); GATv2 is also the only one that reads the edge type.

How the graph is trained. With a contrastive loss (InfoNCE) on *asymmetric* pairs: one view is the complete plan, the other is the same plan with a random 25–75% of its rooms removed, redrawn at every step. For each damaged plan, the network must pick its complete version among the other plans of the batch. This is exactly the task: map an incomplete plan onto the complete one. Recipe: temperature 0.1, batch 256, 600 epochs, training plans only.

Why. with the usual symmetric pairs (two lightly perturbed copies of a plan), every GNN recognised complete plans but failed on damaged ones almost completely: it learned invariance to small perturbations, not to missing rooms.

How the checkpoint is selected ("selected on robustness"). After every epoch we run a small retrieval test: 2,000 validation queries, separate from the evaluation ones, searched against the whole gallery; we keep the epoch with the best robustness. This also tells us there is no overfitting. Overfitting would show as training loss still falling while validation retrieval falls. Instead, in the training log, training and validation loss fall together with a small constant gap, and validation robustness rises and then flattens without a downward trend; in all four replicas the best epoch is at the very end (550–592 of 600). If anything the graph is slightly under-trained, so its score is a lower bound. Random removals redrawn at every step (it never sees the same pair twice), a small model and weight decay protect it. The heads are selected the same way, on 1,000 validation queries; the multi-damage head *did* start to overfit after about 20 epochs (validation robustness down while training loss kept falling) and early stopping kept the best epoch.

Why. select on retrieval, not on the loss: for the graph the validation loss even ranked configurations in the *opposite* order of retrieval. The loss treats every other plan of the batch as a negative, including plans with the same rooms as the anchor, which retrieval would consider close. Selecting on complete plans also failed: it picked early, erratic epochs.

Late fusion (right).

Fusion (paper, Eq. 1)

$$z = [\sqrt{\alpha}\, v\,;\, \sqrt{1-\alpha}\, g], \qquad \langle z_q, z_p \rangle = \alpha \langle v_q, v_p \rangle + (1-\alpha) \langle g_q, g_p \rangle$$

We concatenate the two unit vectors, scaled by $\sqrt{\alpha}$ and $\sqrt{1-\alpha}$. The inner product of two such vectors is the sum of the inner products of their parts, and the square roots on the two sides multiply to α and $1-\alpha$. So the similarity between query q and gallery plan p is a weighted sum of the image cosine and the graph cosine, and the fused vector still has length 1. That is why *one* FAISS search over the 896-d concatenation ranks the gallery by the fused similarity: no need to search the branches separately and

merge rankings. At $\alpha = 1$ or 0 it gives back the single branches, a correctness check.

How α is chosen. On validation, over $0, 0.1, \ldots, 1$. Neighbouring weights often score almost the same, so a plain arg-max would pick by chance. Rule fixed in advance: every weight not significantly worse than the best counts as tied, and we take the median of the tied ones. With the frozen branch this gives 0.4 in all four replicas. Every other fusion picks its own weight with the same rule; the test always uses the weights fixed on validation. "Late" fusion means each branch is trained alone and only their similarities are combined: the branches stay independent, so one can be swapped for a control.

Complementarity gain (bottom right). The key idea. Besides fusing the graph with the image, we fuse it with a second model that holds the *same* information as the graph: a second training of the same network, or a GraphSAGE encoder trained the same way. Whatever these *ensemble controls* gain cannot be new information.

Gain and complementarity gain (paper, Eq. 2)

$$\mathrm{gain}_F(q) = \mathrm{AUC}_F(q) - \mathrm{AUC}_b(q), \qquad \mathrm{CG} = \overline{\mathrm{gain}}_{V+G} - \overline{\mathrm{gain}}_{C^\star}$$

The gain of a fusion is its score minus that of its better branch b, per query. The CG is the image–graph gain minus the gain of the *stronger* control $C^\star$: the part of the gain that a second graph model does not reach. $\mathrm{CG} > 0$: the image brings something of its own. $\mathrm{CG} \leq 0$: the gain is explained by having two models. Taking the stronger control is the conservative choice: it makes it harder to claim that the image helps.

If asked

Q: What is FAISS for, and what does it measure?

It is the search engine that ranks the gallery, not a metric, and it adds no information. It computes the inner product between the query vector and every gallery vector and sorts them from highest to lowest; on unit vectors this is the cosine similarity. The output is a ranking, and every metric is computed on that ranking. So: the encoder produces the embeddings, FAISS ranks by cosine, the metrics judge the ranking.

Q: Is FAISS used both in the single branches and in the fusion?

Yes, the same exact search. For a single branch, one index over that branch's vectors; for the late fusion, the two vectors are concatenated with the $\sqrt{\alpha}$, $\sqrt{1-\alpha}$ weights and one search runs over the concatenated index.

Q: What does the fusion score compute, what does it output, and how does it differ from the FAISS search?

For a query q and a gallery plan p, each branch gives a unit vector: v for the image, g for the graph. The fused vector is $z = [\sqrt{\alpha}\, v\,;\, \sqrt{1-\alpha}\, g]$, and the fusion score is the inner (scalar) product of two fused vectors:

$$\langle z_q, z_p \rangle = \alpha \, \langle v_q, v_p \rangle + (1-\alpha) \, \langle g_q, g_p \rangle$$

Its output is one number: a weighted average of the image cosine and the graph cosine, between -1 and 1; the higher, the more similar the two plans. α sets how much the image counts ($\alpha = 1$: image only; $\alpha = 0$: graph only); the square roots are there because in the inner product $\sqrt{\alpha} \cdot \sqrt{\alpha} = \alpha$. The formula scores *one pair* (q, p); FAISS computes that score between q and all 67,405 gallery plans and sorts them, which gives the ranking on which RR and AUC are computed. Because the score is just the inner product of the concatenated vectors, one exact inner-product search over the 896-d index already ranks by the fused score: no need to search the two branches separately and merge their rankings.

Q: How is α chosen?

It is not trained: each branch is trained on its own and α enters no loss. On validation we try $\alpha = 0, 0.1, \ldots, 1$; for each value we rerun the search for the 2,000 validation queries and measure room-removal robustness AUC, i.e. how high the source comes out. We do not maximise the score itself: a similarity can rise for wrong plans too. Rule fixed in advance: every α not significantly worse

than the best counts as tied, and we take the median of the tied ones. For W + frozen this gives 0.4 in all four replicas; every fusion and control picks its own α with the same rule, and the test uses the values fixed on validation.

Q: Is the damaged query vector a "damaged" vector, or a different vector pointing the same way as the original?

Neither, literally: the query vector is the embedding of the damaged *input*; the damage is never applied to a vector. How close it stays to the original's direction depends on training. *Graph:* InfoNCE teaches the network to bring the query vector close to its source and away from the other plans, so it is "different but pointing the same way". *Frozen vision:* no training on the damage; it is the embedding of a different image and stays close to the original only through visual similarity. *Single-damage head:* an MLP applied to queries and gallery, trained like the graph, so it builds a space where damaged and complete plans coincide better. *Multi-damage head:* acts on the query only, adding a residual correction to the frozen vector that moves it towards the complete plan's frozen vector; the gallery stays frozen.

Q: Do all vision encoders give 768-d vectors? Is the graph always 128-d?

No: 768 is PE-Spatial (a ViT-B); larger encoders give wider vectors; the single-damage head gives 256. Every graph encoder we trained outputs 128.

Q: Why a frozen encoder?

The question is what an image representation adds; a frozen encoder answers it without the confound of training. The heads then test, cheaply, what training on the damage adds.

Q: Why two heads, first a single-damage head and then a multi-damage one?

The single-damage head came first, as the cheapest test of what training on the damage adds: trained on room removal only, it raises the vision branch from 39.6 to 53.3 and the fusion gain more than doubles. But it specialises: it improves room removal and loses on crop and patch (a transfer we had predicted and that failed), and since it transforms the gallery too, it reshapes the whole embedding space. The multi-damage head fixes both limits: trained on the three damages, it improves each of them; acting on the query only, as a correction that starts from the frozen vector, it keeps what the frozen encoder already knows and the gallery does not need to be recomputed. In the fusion with W it gives the largest gain (94.1).

Q: How were the two heads built and trained? What does "query only" versus "query and gallery" mean?

Both are small MLPs on top of the frozen encoder, trained with InfoNCE on training plans only, self-supervised: the positive of a damaged plan is its intact version, no relevance labels. The checkpoint (epoch) is chosen on robustness AUC over 1,000 validation queries.

- *Single-damage head:* $768 \to 1024 \to 256$, applied to queries and gallery (it replaces the vector), room removal only, with whitening re-estimated on its 256-d outputs. It learns to bring the damaged plan close to the complete one; because it also transforms the gallery, it changes the whole space, and specialising on one damage worsens the other two.
- *Multi-damage head:* $768 \to 1024 \to 768$, query only (it adds a correction to the frozen vector; the gallery stays frozen), trained on room removal, crop and patch, no new whitening: the output stays in the space of the frozen vector, which is already whitened. It starts from the identity and only learns to "repair" the damaged query towards the point where the frozen branch puts the intact plan.

"Query only" or "query and gallery" says which vectors pass through the head before the FAISS search: only the query's, or also the other 67k gallery vectors. With the single-damage head every plan, damaged or intact, passes through it, so there is a new 256-d space, and it has to be whitened again: without whitening the cosine would be dominated by a few high-variance directions and by the shared mean, and almost all plans would look alike.

Q: Why is the last layer of the multi-damage head initialised at zero? Is there an explicit residual?

Yes, the residual is explicit: output $= v + \mathrm{MLP}(v)$. With the last layer at zero the head starts as the identity $(v + 0)$, so the query is exactly the frozen vector and training starts from the frozen branch's performance. During training the last layer leaves zero and becomes the correction that moves the damaged query towards the intact plan in the frozen gallery: the head only learns an offset. This matters because the gallery stays frozen, so the query must stay in its space. Without the residual the argument would not hold: a zero last layer would map every query to the same zero vector, a total collapse at the start.

Q: How do you know the training of the GNN and of the heads did not overfit?

After every epoch we measured retrieval on held-out validation queries (2,000 for the graph, 1,000 for the heads, separate from the evaluation queries, against the whole gallery) and kept the best epoch: overfitting would show as training loss still falling while validation retrieval falls. *GNN:* training and validation loss fall together with a small constant gap, validation robustness rises and then flattens with no downward trend, and the best epoch is at the very end (550–592 of 600) in all four replicas; if anything it is slightly under-trained. Removed rooms redrawn at every step, a small model and weight decay protect it, and test values match validation. *Heads:* the multi-damage head did start to overfit after about 20 epochs (validation robustness down, training loss still down) and early stopping kept the best epoch; the single-damage head stopped improving on validation while its validation loss was still falling, so selecting on the loss would have trained it too long. That is why we select on retrieval, not on the loss.

Q: Why two graph models as control?

They hold exactly the graph's information and nothing more, so their gain is the ensemble effect alone.

Slide 4 – Experiments

On the slide. Left: query, metric, protocol, baselines, comparison. Right: test AUC $\times 100$ of single branches and fusions; GATv2-comb beats LayoutGKN by 29 points; every fusion improves on GATv2-comb, mostly at 75% removal.

What to say

Data. RPLAN residential plans: each has a colour rendering (rooms coloured by category) and a record (13 room types, room boxes, typed adjacencies); 4 to 8 rooms per plan. The gallery holds the 67,405 plans with both image and record, in the official training/validation/test split. The gallery always contains every plan, as a search engine would; only what is *learned* (weights, whitening statistics) uses training plans.

Query. A validation or test plan with 25, 50 or 75% of its rooms removed. We remove $\mathrm{round}(f \cdot n)$ of the n rooms: with 7 rooms, 2, 4 and 5. Which rooms is drawn with a seed fixed per query and shared by the two branches, so image and graph lose exactly the same rooms. In the image the removed rooms are painted white and their walls erased; in the graph we drop the removed rooms and their edges, while the remaining rooms keep their original position and size, so the graph describes a plan with holes, not a smaller plan. The vision branch also gets two pixel damages, *crop* (blank a rectangle) and *patch* (blank image patches), used only to choose vision configurations; branches and fusions are compared on room removal, the only damage both can receive.

Metric.

Robustness AUC (paper, Eq. 3)

$$\mathrm{RR}(q,f) = \tfrac{1}{r}\ (r \le 100,\ \text{else } 0)$$

$$\mathrm{AUC}(q) = \frac{1}{3}\sum_{f\in\{.25,.5,.75\}} \mathrm{RR}(q,f), \qquad \mathrm{AUC} = \frac{1}{|Q|}\sum_{q}\mathrm{AUC}(q)$$

For each query and level, the reciprocal rank of the source: 1 if first, 1/2 if second, and so on. We average over the three levels, which is the area under the curve "self-recovery vs fraction removed", then over the queries. 1 means the source is always first; the slide shows it $\times 100$. It follows the metric of on-the-fly sketch retrieval, with "fraction removed" in place of "fraction of the sketch drawn". There is no 0% level: about 3% of RPLAN images have an identical twin, which caps the undamaged score near 0.97 for every system.

Protocol. Every choice (configurations, checkpoints, fusion weights) is made on 2,000 validation queries. Before reading the test split (2,000 queries) we wrote down hypotheses, predicted outcomes and decision rules (*pre-registration*), then read it once with everything fixed. Reason: any choice made after looking at the test set inflates exactly the numbers being reported. Honest caveat: earlier stages of the project, with a weaker graph, also read the test split; nothing reported is selected from those readings.

Statistics. Since all systems see the same queries, we compare two systems query by query: the per-query difference cancels the difficulty both share. For its 95% interval we resample the 2,000 queries with replacement 10,000 times (paired bootstrap); an interval that excludes zero means the difference does not depend on which queries were drawn. Training randomness is covered by repeating the whole pipeline in four replicas with different seeds: an effect counts if it has the same sign in all four and is larger than twice its spread.

Baselines. A room-type histogram without training, and LayoutGKN retrained on our pairs with its own hyperparameters (with random weights it scores essentially zero, so the training matters).

Results (read the bars).

- The histogram finds almost nothing: counting rooms is not enough.
- Vision alone: frozen PE-Spatial 39.6, with the single-damage head 53.3.
- LayoutGKN 61.1; our graph **GATv2-comb** (called W in the paper) 90.3: 29 points more. The gap is at heavy damage: with a quarter of the rooms missing the two are close; with three quarters missing LayoutGKN collapses while ours still ranks the source near the top. Training and selecting for incompleteness makes the difference.
- Every fusion improves on the graph: 91.6 with the frozen image, 93.4 with the single-damage head, 94.1 with the multi-damage head. Almost all of the gain comes at 75% removal, the only level where the graph leaves room to improve.

Transition. A gain of about one point with the frozen image does not tell, by itself, whether the image helps or whether any second model would. Slides 5 and 6 answer it.

If asked

Q: Is LayoutGKN compared fairly?

Retrained on the same damaged–complete pairs, whole training split, its own hyperparameters, scored with our protocol. Trained once, so its value is a lower bound.

Q: The gallery also contains the training and validation plans the models saw. Is that cheating?

No. The target of every query is always a plan the model never trained on: the query is a damaged validation or test plan, and the only relevant item is its intact version in the gallery. Training plans are never the right answer to a test query, so they act only as distractors. This matches real use, as in a search engine that indexes everything. *Weak point:* the comparison between systems is not perfectly symmetric. The frozen vision encoder never saw the training plans, while the GNN and the heads did,

so training plans may be spread differently in the two embedding spaces.

Q: How is removal done in the graph, so that it does not look like the complete graph of a smaller plan?

What tells the query apart from a smaller complete plan is the geometry that remains, and training. Each node carries the centre, width, height, area and aspect ratio of its box on the original 256×256 grid, and the remaining rooms keep them: a damaged plan is "a large plan with holes". An explicit marker exists in the code, the *lost-neighbour marker*, which adds to each remaining room the number of its removed neighbours; but it is not in W, whose nodes have 19 features (the marker would be the twentieth).

Q: Does it change the results that the marker is in some graphs but not in W?

No, and it does not make the comparisons unfair. The marker was one of the factors tried for every encoder, and each encoder kept it only where it improved validation; W does not have it because with GATv2 it made results worse. The numbers of W are those of its best configuration, which has no marker. The GNN comparison is fair: every encoder comes with the best configuration found on validation with the same rule, and no choice was made on test.

Q: In the metric, what are q and f? What does robustness measure, how does it relate to self-recovery, and why rank systems on it?

q is a validation or test plan in its *complete* version: from it we build the damaged query, and the same intact q sits in the gallery as the right answer; Q is the set of query plans, $|Q|$ their number. $f \in \{.25, .50, .75\}$ is the fraction of rooms removed: with n rooms, round$(f \cdot n)$. RR(q, f): damage q at level f, search, find at which rank r the intact q comes out, score $1/r$. AUC(q) averages RR over the three levels: how well q is found, on average, as damage grows. AUC averages over all queries.

Robustness is that AUC: how well a system still finds a plan when it receives only part of it, and how well it holds as the missing part grows. Self-recovery is the task; robustness summarises how self-recovery goes as damage grows. A robustness of 0.9 means that, on average over the three levels, the source is almost always first or close to it. It is computed per damage type (one AUC for room removal, one for crop, one for patch); branches and fusions are compared on room removal, the only damage both receive.

Why rank systems on it: defensible, not beyond question. It holds because it matches the use case and measures exactly finding the plan a fragment comes from; its ground truth is not circular (the only metric in this work without the problem of the relevance axes); and it separates the systems. Its main limit is that it is a proxy: an architect does not look for the source plan, which in reality does not exist, but for any plan compatible with the fragment; self-recovery rewards only the source and penalises a system that ranks other, equally valid plans first. So it is a conservative estimate of real usefulness: a system that cannot find the source is unlikely to find good completions, but finding it does not guarantee that the other results are useful.

Q: Why "points" and not percentages? Is it because duplicates cap the score below 100?

No, the two things are unrelated. The AUC is a mean of reciprocal ranks, between 0 and 1, shown $\times 100$ on the slides; it is *not* the fraction of queries solved. For example 0.5 can mean "half the queries first and half never found", but also "every query second". Differences are absolute differences on the 0–1 scale. The duplicates matter elsewhere: about 3% of RPLAN images have a pixel-identical twin, which ties with the source when the query is intact, so undamaged self-recovery stops near 0.97 for any system. That is why $f = 0$ is excluded: with the query identical to the gallery plan the embedding is the same, the similarity is 1 and the source comes first with any model, except for the twins. Including $f = 0$ would add the same value, about 0.97, to every system, flattening the differences without measuring robustness. Complete plans as queries appear in one place only: the descriptive per-axis relevance analysis (nDCG@10), which asks whether the top 10 plans are similar in topology, composition and geometry.

Q: What role do the per-axis relevance scores play?

Only a descriptive one: a complete plan is the query, and we ask whether the top 10 retrieved plans resemble it on three axes computed from the metadata. They show *what* a system retrieves, not how robust it is. They decide nothing: not W, not the fusion α, not the heads, not the checkpoints. If several graph configurations tie on validation self-recovery AUC, the tie-break uses the topology nDCG, but it never fired. They must be read next to the random floor, already high: 0.740, 0.466 and 0.870 on test.

Q: How are the three axes computed? Is nDCG@10 the only relevance metric?

The axes come from the RPLAN `.mat` records, which hold for each plan the type and box of every room, the adjacencies and the footprint. For each query–gallery pair, each axis gives a similarity in $[0, 1]$:

- *Composition*, "does it have the same rooms?": weighted Jaccard on the 13-bin room-type histograms.
- *Topology*, "are the rooms connected the same way?": weighted Jaccard on the 91-bin histogram where each adjacency is an unordered pair of the 13 types.
- *Geometry*, "same shape and proportions?": the mean of the similarities of footprint area, footprint aspect ratio and per-type area distribution.

Weighted Jaccard measures the overlap of two histograms, $\sum_i \min(a_i, b_i) / \sum_i \max(a_i, b_i)$. The axes are correlated, not independent, so winning on "two axes out of three" is not two separate pieces of evidence; geometry saturates because footprint areas vary little.

nDCG@10 is the only relevance metric reported. nDCG@K measures how good the top K results are, accounting for how relevant each one is and where it appears; it is 1 if the top K are the best possible, in the best order. Three steps: each retrieved plan gets as *gain* its similarity to the query on the chosen axis; DCG sums the gains of the top K, each divided by $\log_2(\text{pos} + 1)$, so a relevant plan counts more higher up; the ideal DCG (IDCG) takes the K most relevant plans of the whole gallery, query excluded, in decreasing order, and nDCG = DCG/IDCG.

Q: What does "circularity" mean here?

The relevance axes are computed from the same information the models receive as input. The graph receives exactly the room types and adjacencies that composition and topology measure, and the footprint is the union of the room boxes; the image shows the room types as 6 colours. If the graph returns plans with a high composition nDCG, it is finding plans whose type histograms match the one it received: almost a comparison of the input with itself. It says the model "reads its own input well" (it preserves the metadata), not that "an architect would find these plans useful"; nobody has checked that similarity on these axes equals usefulness. It also makes the comparison between branches unfair: on composition the graph sees 13 classes and the image 6, so the graph naturally wins. That measures the resolution of the input, not the quality of the model.

Q: What is chosen on validation?

Everything selected, each on robustness:

- *Frozen vision:* encoder, pooling, resolution and whether to whiten, on the mean AUC of the three damages (room removal, crop, patch), 2,000 queries.
- *Heads:* the checkpoint epoch, on robustness over 1,000 queries.
- *Graph:* the factors (temperature, skip connection, rotations and reflections, lost-neighbour marker, depth, width, pooling, aggregator, attention heads), on room-removal AUC over 2,000 queries; then the checkpoint epoch, on room-removal AUC over 2,000 queries disjoint from the evaluation ones.
- *Fusion:* α on the grid (0.4 for W + frozen), on room-removal AUC over 2,000 queries; the controls (W+W, W+SAGE) choose their α with the same rule.

For W the outcome: skip connection kept, rotation and reflection augmentation removed, three layers, width (hidden size) 256, sum pooling, one attention head. Not chosen on validation: the whitening statistics (estimated on training plans), the relevance axes and nDCG.

Q: What are the baselines, and why does nDCG start so high (0.740, 0.466, 0.870)?

Baselines are the reference systems that tell how much our method is worth. There are two: *HIST*, which ranks plans by Euclidean (L2) distance between room-type histograms, the reasonable minimum; and *LayoutGKN*, the closest published method, retrained on our task. The nDCG floor is high because the gain is the similarity on the axis, and two random RPLAN plans are already fairly similar: nearly all plans are flats with about the same rooms, the same connections and the same geometry, so 10 random plans already have a mean gain close to that of the 10 best. The baselines matter mostly for self-recovery: HIST scores 0.001, LayoutGKN 0.611. HIST gets a perfect composition score and fails self-recovery: it ranks by the room-type histogram, exactly what the composition axis measures, so it scores 1.000 by construction.

Q: Does FAISS ranking use the metrics? Is it meaningful to judge a ranking with metrics that did not build it?

FAISS produces one ranking per query, and on it we compute both self-recovery (the rank r of the source, hence RR and AUC) and, in the descriptive analysis, nDCG. The ranking does not use the metrics, on purpose: FAISS ranks by cosine between embeddings, knowing nothing of axes or source plan; the metrics judge that order afterwards. The link goes through training, which makes the cosine a good criterion for the metric we care about: the embeddings of W are optimised for self-recovery (InfoNCE pulls the query towards its source in cosine and pushes away the other plans of the batch), the heads likewise; the frozen vision branch is a pre-trained encoder plus whitening.

So the vectors are optimised to put the source first, not for nDCG, and rightly: nDCG is descriptive and circular, and optimising it would teach the model to reproduce histograms it already has as input. No model would be needed for that: ranking directly by histogram similarity is the HIST baseline, which finds almost nothing in self-recovery.

nDCG does not improve automatically with self-recovery, but it is not independent of it either. The most robust graphs are not the best on complete plans: the GCN combination has a higher topology nDCG@10 than W (0.778 vs 0.647 on validation), while W is the most robust. Both are computed in the same embedding space, though, and a model that finds the source must have learned to put structurally similar plans close, which carries some nDCG with it: W is above the random floor on all three axes without ever being trained for it. The two metrics measure different tasks: self-recovery asks that one plan, the source, be first, starting from a damaged query; nDCG asks that the top 10 be similar to the query on the metadata axes, starting from a complete query. A model can be very good at the first and only fair at the second, like W.

Q: RPLAN renders 13 room types with 6 colours: is that how the dataset is?

Yes. The colour is a fixed function of the room type, but several types share a colour (master room, second room, study, child room, guest room), so the image has a lower resolution than the graph input, where the types are separate.

Slide 5 – Relative strength and ablation

On the slide. Left: the inverted-U plot (fusion gain against graph strength; orange frozen vision, green vision with head, filled points on test). Right: graph ablation (52 → 75 → 91) and vision ablation (raw → whitened). Bottom: the families compared, 37 graph configurations × 4 seeds, 52 vision configurations × 3 damages.

What to say

The plot. Each point is one of the 37 graphs of our configuration search, from the weakest to the final one, fused with a vision branch. On the x axis the AUC of the graph alone, on the y axis the fusion gain. The curve is an inverted U:

- graph much weaker than the image: the image does the work, the graph adds little;
- branches equally strong: the gain peaks, around 20 points, and here even the frozen image passes

the ensemble controls clearly;
- graph near the ceiling: little is left to fix, and the gain shrinks to about one point with the final graph.

The star is an earlier version of our pipeline, with a weaker graph, which measured a gain of about 17 points: it lies on the same curve, so the old large gain and the new small one do not contradict each other. The message: the fusion gain is not a property of the image, it depends on how strong the graph is relative to it. One detail of the green points: with the two weakest graphs the head *lowers* the gain, because there the head becomes the better branch and raises the reference the gain is measured from.

Graph ablation. GCN, GraphSAGE and GATv2 with the same recipe. From a reference configuration we changed one factor at a time (temperature, skip connection, rotation/reflection augmentation, depth, width, pooling...), kept the changes that helped in all four seeds, and combined them; everything on validation. For GATv2, temperature 0.1 alone takes 52 to 75; the combination reaches 91. Combining matters more than any single change, above all at heavy damage. GraphSAGE ends close to GATv2, GCN far behind.

Vision ablation. Eight frozen encoders (DINOv2, DINOv3, I-JEPA, SigLIP2, TIPSv2, C-RADIO, PE-Core, PE-Spatial), 52 configurations, selected on the mean of three damages. Whitening helps every encoder in every setting (PE-Spatial 29 → 52). Newer or larger encoders do not win: TIPSv2, the best on complete plans in our benchmark, ranks sixth under damage. The best configuration changes from damage to damage, which is why we select on their average.

Negative result worth telling (if time allows). Our first room-removal rendering erased the dark outer walls but kept the light inner ones, so each removed room stayed as an empty outlined cell: a free hint. It almost doubled the score of a frozen encoder, and a head learned to exploit it. A pre-registered check on other damages exposed it; this is why vision configurations are selected on three damages.

If asked

Q: Why does the gain peak at equal strength?

Two predictors help each other most when neither dominates; when one is near perfect, the other mostly adds noise and can fix only its few remaining errors.

Q: Was the gain along the curve split into ensemble effect and complementarity with controls, as for W? On all fusions?

No, not on all. The inverted U measures only the raw gain: fused AUC minus the better branch. Controls (graph + second graph, graph + SAGE) exist at three graphs only: W with the frozen branch and the two heads, and the weaker GATv2 configurations *t05* and *ref*, each with the frozen branch and the single-damage head. All other points have only the raw gain. So the curve holds for the raw gain, not for complementarity: the CG was measured on 3 of the 37 graphs and we cannot say it follows an inverted U itself. Where it was measured at the peak, complementarity is real, not only an ensemble effect: CG +8 to +11 points with the frozen branch, up to +15 with the single-damage head.

Q: What is a "configuration" of the graph (*asymrob*, *t05*, *ref*)? Which ones were tried?

A configuration is a set of changes on top of the common recipe of one of the three encoders (GCN, GraphSAGE, GATv2). *ref* is the recipe with no changes; *asymrob* is the graph of the earlier pipeline, the one of the historical gain (the star). One change at a time from *ref*: *t01*, *t02*, *t05* (temperature 0.1, 0.2, 0.5), *noskip*, *nosym* (no rotations and reflections in training), *lost* (removed-rooms marker, a twentieth feature), *l3* (three layers), *h256* (width 256), *pmm* (mean + max pooling instead of sum), *amax*/*aadd* (aggregator), *hd1*/*hd8* (1 or 8 attention heads). *comb* combines the changes that gave a net gain in stage 1, keeping the best per factor; GATv2-comb (W) is *t01 + nosym + l3 + h256 + hd1*. Configurations and combinations of the three encoders give 37 graphs.

Q: How was the winning configuration chosen for each branch?

Always on robustness on validation, but the criterion changes from branch to branch: room-removal AUC for the graph (37 configurations × 4 seeds); the mean AUC of the three damages for vision (8

encoders, 52 configurations); room-removal AUC for the fusion α.

Q: Does the single-damage head work on other damages?

No: it improves room removal and loses on crop and patch. A head learns the damage it sees, which is why we built the multi-damage head.

Slide 6 – Complementarity gain

On the slide. Table on the test split, mean of four replicas, in AUC points: fusion gain, control gain, CG and outcome for the three vision branches fused with W. Below: frozen vision, damage-trained heads, where the image helps.

What to say

How to read the table. Fusion gain: fused AUC minus the better branch. Control gain: the same for the stronger control, here a second training of W fused with W. CG: the difference (paper, Eq. 2).

The control. Fusing the graph with a second copy of itself already gains +1.63 points. That is the ensemble effect alone: no new information.

Frozen vision. Gains +1.25, *less* than the control: CG −0.38, negative in all four replicas. Against the GraphSAGE control it is equivalent (the difference is within the ±0.5-point margin we fixed). So with a strong graph the frozen image encoder adds nothing beyond an ensemble effect, while the raw +1.25 alone would have suggested it helps.

Damage-trained heads. They gain +2.79 and +3.66, exceeding the control by +1.16 and +2.03, positive in all four replicas: they carry information of their own. But even here the control explains about half of the gain, so the raw fusion gain overstates the image's contribution by roughly a factor of two.

Where the image helps. Splitting the queries: the CG is zero with 25 or 50% removed and becomes positive only at 75%; it is largest on small plans (five rooms or fewer), where even the frozen branch passes the control. Reason: with few rooms left, the partial graph is compatible with many plans, while the image still keeps the shape of the hole. We also expected a larger CG for plans whose room graph is shared by many others; the data refuted it. This analysis is post hoc and on one seed: a description, not a confirmed result. It is consistent with slide 5: with graphs only as strong as the image, even the frozen branch passes the controls by 8 to 16 points.

If asked

Q: Why is the frozen CG negative?

With a graph near the ceiling, a second graph model corrects more of its remaining errors than the frozen image does.

Q: Do control gain and CG add up to the fusion gain?

Only as a summary: they come from two different fusions, so they describe the gain but do not split one fusion exactly.

Slide 7 – Conclusions

On the slide. Graph branch, complementarity, recommendation; limitations and future work.

What to say

Graph branch. Trained on partial plans and selected on robustness, the graph retrieves a plan from half of its rooms almost as well as from three quarters (95.4 vs 96.3 points at 50% and 25% removal). *Careful:* the slide (and the paper) say "as well as from the complete plan", but we never measure the

complete plan (no 0% level); say "with a quarter missing".

Complementarity. The image adds information of its own only under two conditions: it is trained on the damage, or the graph is no stronger than the image.

Recommendation, beyond floor plans. Part of any fusion gain comes just from combining two models. Compare it with an ensemble control, here a fusion of two graph models, to measure what the second modality really adds.

Limitations. RPLAN only, with synthetic damages. Graphs come from exact records, not parsed from images, which also makes the relevance scores circular. Training budgets bind: the graph's best epochs are at the end of training, LayoutGKN was trained once, so both scores are lower bounds. One global fusion weight. The multi-damage head was trained once and tested only on the damages it saw. The "where it helps" analysis is post hoc, on one seed.

Future work. Graphs parsed from images, external datasets, a learned or per-query fusion, a damage the heads never saw, human relevance judgements.